

Analyse des algorithmes

<https://www.developpement-informatique.com>

Mr. ESSADDOUKI Mostafa
Professeur agrégé en informatique



■ Suite de Fibonacci:

La suite de **Fibonacci** est définie par :

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{pour } n \geq 2$$

- 1 Écrire une fonction *réursive* ***fib_rec(n)*** qui retourne le terme d'indice n de la suite de Fibonacci.
- 2 Écrire une fonction *itérative* ***fib_iter(n)*** qui retourne le terme d'indice n de la suite de Fibonacci.

Deux programmes peuvent résoudre le même problème, mais avec des performances très différentes.

Deux programmes peuvent résoudre le même problème, mais avec des performances très différentes.

☞ *Exemple(s) 1.2: Recherche d'un élément dans un tableau*

- ❑ **Méthode 1** : parcourir le tableau élément par élément → recherche linéaire.
- ❑ **Méthode 2** : diviser le tableau en deux à chaque étape (si le tableau est trié) → recherche dichotomique.

L'analyse des algorithmes vise donc à mesurer l'efficacité et à comparer les approches.

Ainsi, analyser un algorithme, c'est mesurer sa consommation de ressources afin de pouvoir :

- ❑ Comparer deux méthodes de résolution.
- ❑ Choisir la plus adaptée en fonction du contexte (temps, mémoire, taille des données).
- ❑ Prévoir le comportement de l'algorithme sur de grandes données, sans dépendre de la machine utilisée.

Notations asymptotiques

Au lieu de mesurer le temps exact d'exécution (qui dépend des machines), on utilise des notations asymptotiques qui permettent de donner une estimation générale de la croissance de la fonction de complexité.

Ces notations donnent une idée de l'ordre de grandeur de la complexité : elles ignorent les constantes et les termes de moindre importance.

Notations asymptotiques

Au lieu de mesurer le temps exact d'exécution (qui dépend des machines), on utilise des notations asymptotiques qui permettent de donner une estimation générale de la croissance de la fonction de complexité.

Ces notations donnent une idée de l'ordre de grandeur de la complexité : elles ignorent les constantes et les termes de moindre importance.

☞ *Exemple(s) 1.4:*

$$f(n) = 5n^2 + 3n + 10 \quad \text{se note en } O(n^2)$$

Définition 1

On dit que $f(n) = O(g(n))$ quand il existe une constante c_1 telle que $f(n) \leq c_1 \cdot g(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

Définition 2

On dit que $f(n) = O(g(n))$ quand il existe une constante c_1 telle que $f(n) \leq c_1 \cdot g(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

La notation O exprime la vitesse maximale de croissance d'un algorithme : elle indique que le temps d'exécution ne dépassera jamais une certaine fonction.

👉 Exemple(s) 1.5:

$$f(n) = 3n^2 + 2n + 1, \quad g(n) = n^2$$

Pour $n \geq 1$:

$$f(n) \leq 6n^2$$

On peut choisir $c = 6$ et $n_0 = 1$. $\rightarrow$ Donc $f(n) = O(n^2)$.

👉 Exemple(s) 1.6:

$$f(n) = 7n \log n + 20n, \quad g(n) = n \log n$$

Pour $n \geq 2$, $\log n \geq 1$, donc :

$$f(n) \leq (7 + 20) n \log n = 27n \log n$$

On peut choisir $c = 27$, $n_0 = 2$. $\rightarrow$ Donc $f(n) = O(n \log n)$.

Définition 3

On dit que $f(n) = \Omega(g(n))$ quand il existe une constante c_1 telle que $c_1 \cdot g(n) \leq f(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

Définition 4

On dit que $f(n) = \Omega(g(n))$ quand il existe une constante c_1 telle que $c_1.g(n) \leq f(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

La notation Ω fixe une limite minimale : elle garantit qu'un algorithme prendra au moins autant de temps qu'une certaine fonction.

👉 *Exemple(s) 1.7:*

$$f(n) = 3n^2 + 2n + 1, \quad g(n) = n^2$$

Pour $n \geq 1$:

$$f(n) \geq 3n^2$$

On peut choisir $c = 3$, $n_0 = 1$. $\rightarrow$ Donc $f(n) = \Omega(n^2)$.

👉 *Exemple(s) 1.8:*

$$f(n) = n \log n + n, \quad g(n) = n \log n$$

Pour $n \geq 2$, on a $\log n \geq 1$, donc :

$$f(n) \geq n \log n$$

On peut choisir $c = 1$, $n_0 = 2$. $\rightarrow$ Donc $f(n) = \Omega(n \log n)$.

Définition 5

On dit que $f(n) = \Theta(g(n))$ quand il existe des constantes c_1 et c_2 telles que $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

Définition 6

On dit que $f(n) = \Theta(g(n))$ quand il existe des constantes c_1 et c_2 telles que $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ pour toutes les valeurs suffisamment grande de n . Ici, n est un entier positif.

La notation Θ encadre la croissance d'un algorithme : elle affirme que l'algorithme croît au même rythme qu'une certaine fonction, ni plus vite ni moins vite.

👉 Exemple(s) 1.9:

$$f(n) = 3n^2 + 2n + 1, \quad g(n) = n^2$$

Pour $n \geq 1$:

$$3n^2 \leq f(n) \leq 6n^2$$

On peut choisir $c_1 = 3$, $c_2 = 6$, $n_0 = 1$. $\rightarrow$ Donc $f(n) = \Theta(n^2)$.

Exemple(s) 1.10:

$$f(n) = 4n \log n + 7n, \quad g(n) = n \log n$$

Pour $n \geq 2$:

$$4n \log n \leq f(n) \leq 11n \log n$$

On peut choisir $c_1 = 4$, $c_2 = 11$, $n_0 = 2$. $\rightarrow$ Donc $f(n) = \Theta(n \log n)$.

👉 Exemple(s) 1.11:

$$f(n) = 2n^3 + 5n^2, \quad g(n) = n^3$$

Pour $n \geq 1$:

$$2n^3 \leq f(n) \leq 3n^3$$

On peut choisir $c_1 = 2$, $c_2 = 3$, $n_0 = 1$. $\rightarrow$ Donc $f(n) = \Theta(n^3)$.

Notion de complexité algorithmique

La complexité d'un algorithme mesure la quantité de ressources qu'il consomme en fonction de la taille de ses données d'entrée.

La complexité d'un algorithme mesure la quantité de ressources qu'il consomme en fonction de la taille de ses données d'entrée.

On distingue deux aspects principaux :

- ❑ **Complexité temporelle** : temps d'exécution (nombre d'opérations élémentaires effectuées).
- ❑ **Complexité spatiale** : mémoire supplémentaire nécessaire (hors stockage des données d'entrée).

👉 Exemple(s) 1.12:

```
def somme(tab):  
    s = 0  
    for x in tab:  
        s += x  
    return s
```

- ❑ Temps d'exécution : proportionnel au nombre d'éléments $n \rightarrow O(n)$.
- ❑ Espace mémoire : uniquement la variable s et l'itérateur $\rightarrow O(1)$.

L'analyse de la complexité cherche à évaluer le temps d'exécution d'un algorithme indépendamment de la machine.

En effet, si l'on tient compte de paramètres matériels (fréquence du processeur, nombre de cœurs, temps d'accès disque. . .), la mesure devient trop spécifique et impraticable.

Pour simplifier, on adopte un modèle théorique de calcul qui fixe des règles et permet de comparer les algorithmes de façon abstraite.

Modèle RAM (Random Access Machine)

Le modèle de référence utilisé est celui d'une machine séquentielle à accès aléatoire :

- ❑ La mémoire est composée de cases accessibles en temps constant (accès direct par adresse).
- ❑ Les instructions sont exécutées une à une, sans parallélisme.
- ❑ Chaque opération élémentaire a un coût constant (hypothèse du coût unitaire).

Opérations élémentaires

Une opération élémentaire est une action de base dont le temps d'exécution est considéré constant.

- ❑ Lecture ou écriture dans une case mémoire.
- ❑ Affectation d'une variable.
- ❑ Comparaison entre deux entiers ou réels.
- ❑ Opérations arithmétiques simples : addition, soustraction, multiplication, division entière, modulo.

Exemple(s) 1.13:

| $c = a + b$

- ☐ Lecture de a
- ☐ Lecture de b
- ☐ Addition
- ☐ Écriture du résultat dans c

Total : 4 opérations élémentaires.

- ❑ **Hypothèse** : un entier (de n'importe quelle taille) tient dans une seule case mémoire.
- ❑ Chaque opération arithmétique sur un entier ou flottant est exécutée en temps constant.
- ❑ Très utilisé car proche du fonctionnement des processeurs actuels (opérations rapides sur 32 ou 64 bits).

👉 *Exemple(s) 1.14:*

```
def addition(x, y):  
    return x + y
```

Temps d'exécution constant $O(1)$.

- ❑ **Hypothèse plus réaliste** : le temps dépend de la taille en bits des nombres manipulés.
- ❑ Plus les nombres sont grands, plus les calculs nécessitent de ressources.

👉 *Exemple(s) 1.15:*

```
# concaténation de chaînes ou tableaux
def concatener(x, y):
    return x + y
```

- ✓ Nécessite d'allouer une nouvelle mémoire et de copier tous les caractères.
- ✓ Complexité temporelle : $O(\text{len}(x) + \text{len}(y))$

Ici, le coût unitaire ne serait pas adapté : il faut prendre en compte la taille des données.

Lorsqu'on analyse un algorithme, il n'a pas toujours le même comportement selon l'entrée.

- ❑ Parfois, il termine très vite.
- ❑ Parfois, il prend beaucoup plus de temps.

👉 *Exemple(s) 1.16:*

Recherche d'un élément dans un tableau

- ❑ Si l'élément est en première position → très rapide.
- ❑ Si l'élément est en dernière position ou absent → beaucoup plus lent.

C'est pourquoi on distingue trois situations :

- ❑ **Pire cas** : le scénario le plus défavorable.
- ❑ **Meilleur cas** : le scénario le plus favorable.
- ❑ **Cas moyen** : comportement “typique” (moyenne sur toutes les entrées).

Définition 7

Le **pire cas** correspond au nombre **maximal** d'opérations que l'algorithme peut effectuer pour une entrée de taille n .

- ✓ Donne une **borne supérieure** du temps d'exécution.
- ✓ Utile pour garantir que l'algorithme ne dépassera pas un certain temps, quelle que soit l'entrée.

👉 Exemple(s) 1.17: Recherche linéaire

```
def recherche(tab, x):  
    for i in range(len(tab)):  
        if tab[i] == x:  
            return i  
    return -1
```

- ❑ Pire cas : x n'est pas présent $\rightarrow$ on parcourt tout le tableau.
- ❑ Nombre de comparaisons = n .
- ❑ Complexité = $O(n)$.

Définition 8

Le **meilleur cas** correspond au nombre **minimal** d'opérations que l'algorithme peut effectuer pour une entrée de taille n .

- ✓ Indique le temps d'exécution dans les conditions les plus favorables.
- ✓ Peu utile en pratique, car ce cas se produit rarement.

👉 Exemple(s) 1.18: Recherche linéaire

- ❑ Meilleur cas : x est le premier élément.
- ❑ Nombre de comparaisons = 1.
- ❑ Complexité = $O(1)$.

Définition 9

Le **cas moyen** correspond au **nombre moyen** d'opérations effectuées, en considérant que toutes les entrées de taille n sont équiprobables (même probabilité).

- ✓ Représente le **comportement typique** de l'algorithme.
- ✓ Plus réaliste que le meilleur cas, mais souvent difficile à calculer (il faut connaître la distribution des entrées).

👉 *Exemple(s) 1.19: Recherche linéaire*

Supposons que l'élément recherché a la même probabilité d'être à n'importe quelle position (ou absent).

❑ Moyenne :

$$T(n) = \frac{1 + 2 + 3 + \cdots + n + (n + 1)}{n + 1}$$

$$T(n) = \frac{(n + 1)(n + 2)}{2(n + 1)} \approx \frac{n}{2}$$

❑ Complexité = $O(n)$.

Méthode de comptage des pas

La **méthode de comptage des pas** est l'une des techniques les plus utilisées pour analyser la complexité temporelle d'un algorithme.

Elle consiste à **compter combien de fois chaque instruction est exécutée**, puis à sommer les coûts pour obtenir une fonction $T(n)$ dépendant de la taille n de l'entrée.

L'objectif est de déterminer l'**ordre de grandeur** de $T(n)$, c'est-à-dire son comportement asymptotique.

■ Principe de la méthode:

- 1 Identifier les instructions élémentaires (lecture, affectation, comparaison, etc.).
- 2 Compter combien de fois chaque instruction est exécutée, en fonction de n .
- 3 Multiplier par le coût de l'instruction (souvent supposé constant).
- 4 Additionner tous les coûts pour obtenir la fonction $T(n)$.
- 5 Simplifier (on ne garde que le terme dominant).

Règle maximale

Si un algorithme est constitué de plusieurs blocs exécutés séquentiellement, le temps total est dominé par le bloc le plus coûteux.

$$T(n) = \max(T_A(n), T_B(n), \dots)$$

👉 Exemple(s) 1.20:

```
if condition:
    # bloc A : O(1)
else:
    # bloc B : O(n)
```

Complexité totale = $\max(O(1), O(n)) = O(n)$

■ Cas des instructions simples:

Une instruction simple (sans boucle ni appel récursif) a une complexité constante.

👉 *Exemple(s) 1.21:*

```
a = 2  
b = 3 + 4  
c = a*b - 2
```

Chacune est exécutée une seule fois $\rightarrow O(1)$

■ Cas des appels de fonctions:

- ❑ Si la fonction contient uniquement des instructions simples $\rightarrow O(1)$.
- ❑ Si elle contient une boucle ou un appel récursif $\rightarrow$ la complexité de la fonction est celle de son corps.

👉 *Exemple(s) 1.22:*

```
def fct3(a):    # boucle sur n
    for i in range(len(a)):
        print(i)
```

Complexité = $O(n)$

■ Structures conditionnelles (if – else – elif):

On applique la règle maximale :

$$T(n) = \max(T(\text{branche1}), T(\text{branche2}), \dots)$$

👉 Exemple(s) 1.23:

```
if cond1:           # O(1)
    ...
elif cond2:         # O(n^2)
    ...
else:               # O(n)
    ...
```

Complexité = $\max(O(1), O(n^2), O(n)) = O(n^2)$

■ Boucle for:

Le coût est proportionnel au nombre d'itérations.

👉 *Exemple(s) 1.24:*

```
for i in range(n):  
    print(i)
```

- ✓ Initialisation : 1 fois.
- ✓ Comparaison : $n+1$ fois.
- ✓ Incrémentation : n fois.
- ✓ Instruction : n fois.

Complexité totale = $O(n)$

■ Boucle for:

👉 *Exemple(s) 1.25:*

```
def max_val(tab):  
    m = tab[0]                # 0(1)  
    for i in range(len(tab)): # n itérations  
        if tab[i] > m:        # n comparaisons  
            m = tab[i]        # au plus n affectations  
    return m                  # 0(1)
```

Complexité = $O(n)$

■ Boucle while:

Pour une boucle for, le nombre d'itérations est généralement explicite (ex. **for i in range(n)** → exactement **n** tours).

Mais pour une boucle while, il faut estimer le nombre d'itérations :

- 1 Identifier la variable de contrôle (celle qui change à chaque tour).
- 2 Étudier la condition d'arrêt.
- 3 Déterminer comment évolue la variable de contrôle.
- 4 En déduire le nombre de tours possibles.

■ Boucle while:

👉 *Exemple(s) 1.26:*

```
i = 0
while i < n:
    i += 1
```

- ✓ Variable de contrôle : 'i'.
- ✓ Condition d'arrêt : 'i < n'.
- ✓ Évolution : 'i' augmente de 1 à chaque tour.
- ✓ Nombre d'itérations $\approx n$.

Complexité = $O(n)$

■ Boucle while:

👉 *Exemple(s) 1.27:*

```
i = 1
while i < n:
    i *= 2
```

- ✓ Variable de contrôle : i .
- ✓ Condition d'arrêt : $i < n$.
- ✓ Évolution : 'i' double à chaque tour.
- ✓ Après k tours : $i = 2^k$.
- ✓ La boucle s'arrête quand $2^k \geq n$.

$$k \approx \log_2(n)$$

Complexité = $O(\log n)$

■ Boucle while:

👉 *Exemple(s) 1.28:*

```
i = n
while i > 0:
    i -= 1
```

- ✓ Variable de contrôle : 'i'.
- ✓ Condition d'arrêt : 'i > 0'.
- ✓ Évolution : 'i' diminue de 1 à chaque tour.
- ✓ Nombre de tours = n .

Complexité = $O(n)$

■ Boucle while:

👉 Exemple(s) 1.29:

```
i = n
while i > 1:
    i //= 2
```

- ✓ Variable de contrôle : 'i'.
- ✓ Condition d'arrêt : 'i > 1'.
- ✓ Évolution : 'i' est divisé par 2 à chaque tour.
- ✓ Après k tours : $i = n/2^k$.
- ✓ La boucle s'arrête quand $n/2^k \leq 1$.

$$k \approx \log_2(n)$$

Complexité = $O(\log n)$

■ Boucle while:

👉 Exemple(s) 1.30:

```
p = 0
i = 0
while p < n:
    p += i
    i += 1
```

✓ Variables de contrôle : 'p' et 'i'.

✓ Évolution :

❑ 'i' croît linéairement.

❑ 'p' croît comme la somme

$$0 + 1 + 2 + \dots + i = i(i+1)/2.$$

✓ Condition d'arrêt : $p \geq n$.

✓ Arrêt quand $i^2 \approx n$.

$$i \approx \sqrt{n}$$

Complexité = $O(\sqrt{n})$

■ Boucles imbriquées:

- 1 Identifier le nombre d'itérations de chaque boucle.
- 2 Écrire la **somme** exacte.
- 3 Simplifier avec des sommes connues : $\sum_{i=1}^n 1 = n$; $\sum_{i=1}^n i = \frac{n(n+1)}{2}$;
 $\sum_{i=1}^n \log i = O(n \log n)$; $\sum_{k=0}^{\lfloor \log_2 n \rfloor} 1 = O(\log n)$.
- 4 Conclure en $O(\cdot)$.

■ Boucles imbriquées:

👉 *Exemple(s) 1.31:*

```
for i in range(n):  
    for j in range(n):  
        op()
```

Pour chaque i , la boucle interne exécute exactement n opérations $O(1)$. Somme :

$$\sum_{i=1}^n \sum_{j=1}^n 1 = n \cdot n = n^2.$$

→ Complexité = $O(n^2)$

■ Boucles imbriquées:

👉 Exemple(s) 1.32:

```
for i in range(n):  
    for j in range(i,n):  
        op()
```

Pour un i fixé : nombre d'itérations $= (n - 1) - (i) + 1 = n - i$.

$$\text{Somme : } \sum_{i=1}^n (n - i) = \underbrace{\sum_{i=1}^n n}_{=n^2} - \underbrace{\sum_{i=1}^n i}_{=n(n+1)/2}$$

$$= n^2 - \frac{n(n+1)}{2} = \frac{n(n-1)}{2}.$$

$$\rightarrow \text{Complexité} = O(n^2)$$

■ Boucles imbriquées:

👉 *Exemple(s) 1.33:*

```
for i in range(n):  
    for j in range(i):  
        op()
```

Somme : $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

→ Complexité = $O(n^2)$

■ Boucles imbriquées:

👉 *Exemple(s) 1.34:*

```
for i in range(n):  
    j = 1  
    while j <= n:  
        op()  
        j = 2*j
```

Pour un i fixé : $\lfloor \log_2 i \rfloor + 1 = O(\log i)$ itérations.

Somme : $\sum_{i=1}^n O(\log i) = O\left(\sum_{i=1}^n \log i\right) = O(n \log n)$.

(En effet $\sum_{i=1}^n \log i = \log(n!) = O(n \log n)$, Stirling.)

→ Complexité = $O(n \log n)$

Classes de complexité algorithmique

L'étude de la complexité algorithmique permet de comparer les performances des algorithmes indépendamment de la machine utilisée.

On distingue principalement des **classes de complexité** qui regroupent les algorithmes selon leur ordre de grandeur en fonction de la taille de l'entrée n .

■ Complexité logarithmique – $O(\log n)$:

- ✓ Exemple : recherche dichotomique.
- ✓ Très efficace même pour de grandes valeurs de n .
- ✓ Croissance très lente.

■ Complexité linéaire – $O(n)$:

- ✓ Exemple : recherche séquentielle dans un tableau.
- ✓ Temps proportionnel à la taille des données.

■ Complexité quasi-linéaire – $O(n \log n)$:

- ✓ Exemple : tri par fusion, tri rapide (en moyenne).
- ✓ Considérée comme la limite des algorithmes « rapides » pour le tri.

■ Complexité polynomiale – $O(n^k)$:

- ✓ Exemple : algorithme de Floyd-Warshall ($O(n^3)$).
- ✓ Si k est petit (≤ 2 ou 3), reste exploitable.
- ✓ Pour $k > 3$, devient souvent lent en pratique.

■ Complexité exponentielle – $O(2^n)$, $O(n!)$:

- ✓ Exemple : exploration exhaustive des permutations (problème du voyageur de commerce).
- ✓ Très rapidement impraticable dès que n dépasse 20 ou 30.

⇒ **Hierarchie et comparaison:**

n	$\log_2(n)$	n	$n \log_2(n)$	n^2	n^3	n^4	2^n
10	3.32	10	33.22	100	1000	10000	1024
20	4.32	20	86.4	400	8000	160000	1 048 576
30	4.91	30	147.3	900	27 000	810 000	1.07×10^9

On remarque que $O(\log n)$ et $O(n)$ restent très raisonnables, tandis que $O(2^n)$ explose très vite.

Classes de complexité algorithmique

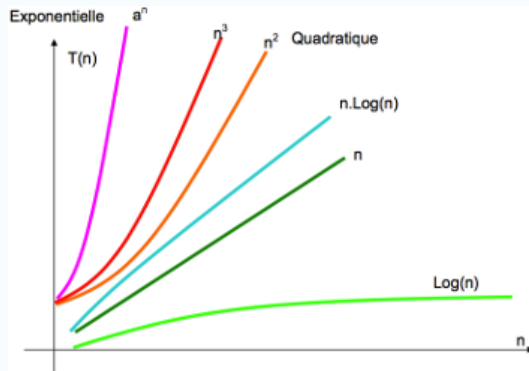


Figure 1 – Comparaison asymptotique des complexités

Lorsqu'on étudie un algorithme, on ne se limite pas au temps d'exécution, mais aussi à la mémoire supplémentaire qu'il consomme.

Définition 10

Par complexité spatiale, on entend la mémoire auxiliaire utilisée par l'algorithme en plus de celle nécessaire au stockage des données d'entrée.

■ Composantes de la mémoire auxiliaire:

En Python, la mémoire supplémentaire se décompose en :

☐ Variables simples

- ➡ Exemples : compteurs, booléens, valeurs temporaires.
- ➡ Mémoire constante $O(1)$

☐ Structures de données auxiliaires

- ➡ Listes, dictionnaires, ensembles créés par l'algorithme.
- ➡ Leur taille dépend de n

☐ Pile d'appels récursifs

- ➡ Chaque appel récursif en Python ajoute un cadre d'exécution.
- ➡ Profondeur = dépend du nombre d'appels récursifs.

👉 Exemple(s) 1.35: Recherche d'un élément dans une liste

```
def recherche(liste, x):  
    for elem in liste:  
        if elem == x:  
            return True  
    return False
```

- ✓ Variables : $\text{elem}, x \rightarrow O(1)$.
- ✓ Pas de structure supplémentaire.

→ Complexité spatiale : $O(1)$

👉 Exemple(s) 1.36: Création d'une nouvelle liste filtrée

```
def pairs(liste):  
    return [x for x in liste if x % 2 == 0]
```

- ✓ Nouvelle liste créée contenant au plus n éléments.
- ✓ Pas de structure supplémentaire.

→ Complexité spatiale : $O(n)$

👉 Exemple(s) 1.37: Récursion naïve de Fibonacci

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

- ✓ Profondeur de récursion : n
- ✓ Pas de structure supplémentaire.

→ Complexité spatiale : $O(n)$ (pile d'appels).

👉 Exemple(s) 1.38: Version itérative de Fibonacci

```
def fib_iter(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a+b  
    return a
```

✓ Variables $a, b, _ \rightarrow O(1)$

→ Complexité spatiale : $O(1)$